



Zomato Food Delivery Time Prediction

Graduation Project — Advanced Data Analysis Course

Regression Track

Team Members

Menna Khaled — Feature Engineering, Main Regression Model, Streamlit Application, Project Coordination

Abdelrahman Mahmoud — Data Preprocessing, Exploratory Data Analysis, Baseline Linear Regression

Dareen Alaa — Data Cleaning Lead, Decision Tree Regression

Salma Tarek — Power BI Data Modeling, DAX Measures & Relationships

Moureen Ramy Kelada — Power BI Dashboard Development (all four pages)

September 2026

1. Abstract

Food delivery services have become an important part of modern food businesses, where accurate delivery-time estimation can improve customer satisfaction and support better operational planning. This project focuses on developing a machine learning system capable of predicting food delivery time based on different operational and environmental factors.

The project uses the Zomato Delivery Operations Analytics Dataset and follows a complete data science workflow, including data cleaning, exploratory data analysis, feature engineering, machine learning, model evaluation, interactive visualization, and application deployment.

Three regression models were developed and evaluated: Linear Regression, Decision Tree Regression, and a tuned Random Forest Regression model. The models were evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 . The tuned Random Forest achieved the best performance (MAE = 3.22 minutes, RMSE = 4.08 minutes, $R^2 = 0.807$) and was selected as the deployed model. An interactive Streamlit application was developed to allow users to enter delivery-related information and obtain an estimated delivery time, and a four-page Power BI dashboard was built to communicate operational findings.

The final project combines predictive machine learning with interactive dashboards and a deployed web application to demonstrate how data-driven techniques can be applied to food delivery operations.

2. Introduction

Food delivery platforms handle a large number of orders every day, making accurate delivery-time estimation an important operational challenge. Delivery time can be affected by several factors, including the distance between the restaurant and the customer, traffic conditions, weather, order volume, and the time at which an order is placed.

Inaccurate delivery-time estimates may lead to customer dissatisfaction and difficulties in managing delivery operations. Therefore, using historical delivery data to identify important patterns and build predictive models can help provide more reliable delivery-time estimates.

This project applies data analysis and machine learning techniques to the Zomato Delivery Operations Analytics Dataset. The project starts with data preparation and cleaning, followed by exploratory data analysis to understand the relationships between delivery time and different factors. Feature engineering is then performed to create additional useful features, such as geographical distance and time-based features.

After preparing the data, three regression models are developed and evaluated. The project also includes an interactive Power BI dashboard for analyzing delivery patterns and a Streamlit application that allows users to generate delivery-time predictions using the trained machine learning model.

The overall aim is to demonstrate a complete end-to-end machine learning workflow, from raw data preparation to model development, visualization, and deployment.

3. Business Problem

The main business problem addressed by this project is the difficulty of accurately estimating food delivery time. Food delivery time may vary depending on several factors, such as:

- Distance between the restaurant and delivery location
- Traffic conditions

- Weather conditions
- Number of parallel orders handled by a driver
- Time of day and day of week
- Festival periods
- Other operational factors (vehicle type, vehicle condition, delivery-person characteristics)

A machine learning model can use historical data to learn relationships between these factors and actual delivery time. This can be used to generate estimated delivery times for new delivery situations.

Main Business Question

How can machine learning be used to predict food delivery time based on operational, geographical, environmental, and time-related factors?

4. Project Objectives

- Prepare and clean the Zomato delivery dataset by handling missing values, invalid values, incorrect data types, and outliers where necessary.
- Perform exploratory data analysis (EDA) to identify patterns and relationships between delivery time and factors such as traffic, weather, order volume, and time.
- Perform feature engineering by creating useful features such as restaurant-to-delivery distance, hour of day, day of week, and peak-hour indicators.
- Develop machine learning regression models for predicting food delivery time.
- Tune selected machine learning models to improve their predictive performance.
- Evaluate and compare the developed models using MAE, RMSE, and R^2 .
- Develop an interactive Power BI dashboard to visualize geographical, weather, traffic, and time-related delivery patterns.
- Develop and deploy a Streamlit application that allows users to enter delivery information and obtain a predicted delivery time.
- Combine data analysis, machine learning, visualization, and deployment into one complete end-to-end project.

5. Dataset Description

The project uses the Zomato Delivery Operations Analytics Dataset, obtained from Kaggle.

Dataset Source: Kaggle

Dataset Name: Zomato Delivery Operations Analytics Dataset

Dataset URL: <https://www.kaggle.com/datasets/saurabhbadole/zomato-delivery-operations-analytics-dataset>

Access Date: September 18, 2026

Original size: 45,584 orders and 20 columns.

Cleaned size (used for EDA and modeling): 45,314 orders and 22 columns, after removing 270 rows with an outlier target value during data cleaning (see Section 9).

The dataset contains information related to food delivery operations, including delivery-person details, restaurant and delivery coordinates, order and pickup timestamps, weather, traffic, vehicle information, festival status, city type, and delivery ratings.

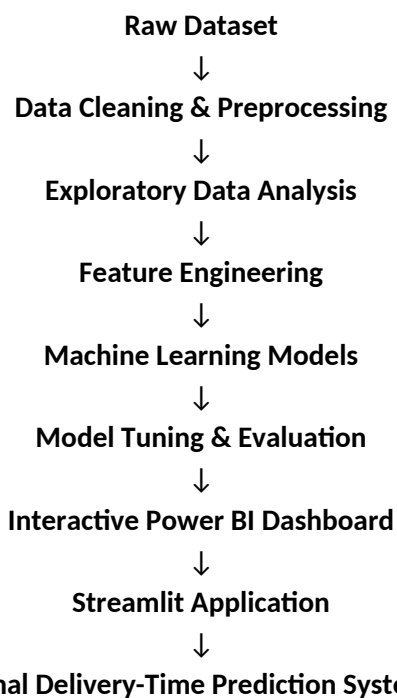
Target variable: Time_taken (min) — the delivery duration that the machine learning models are designed to predict.

Dataset Files

File	Description
data/Zomato Dataset.csv	Original, uncleaned dataset (45,584 rows, 20 columns)
data/Zomato_Cleaned_Dareen.csv	Cleaned dataset used for EDA and modeling (45,314 rows, 22 columns)

6. Project Methodology

The project follows an end-to-end machine learning workflow:



Each stage of the workflow is described in detail in the following sections.

7. Project Structure

The project is organized into several main components:

Data Preparation: The raw dataset is cleaned and prepared for analysis and machine learning.

Exploratory Data Analysis: Statistical analysis and visualizations are used to understand the dataset and identify important patterns.

Feature Engineering: Additional predictive features are created from the available geographical and time-related information.

Machine Learning: Three regression models are trained and evaluated for delivery-time prediction.

Dashboard: Interactive Power BI visualizations are developed to analyze delivery patterns related to geography, weather, traffic, and time.

Streamlit Application: A deployed web application allows users to enter delivery-related information and receive a predicted delivery time.

Documentation and Presentation: The project findings, methodology, results, limitations, and applications are documented and presented as part of the final project submission.

8. Data Preprocessing and Cleaning

Data preprocessing was performed to prepare the Zomato Delivery Operations Analytics Dataset for exploratory analysis and machine learning. The original dataset contained 45,584 rows and 20 columns. The preprocessing stage focused on data quality, consistency, validity, and the creation of usable time-based features.

8.1 Initial Data Inspection

The dataset was first inspected to understand its structure, data types, column names, sample records, and overall dimensions. Column names were standardized by removing leading and trailing spaces and normalizing repeated whitespace.

8.2 Missing Values and Placeholder Values

Missing-value analysis identified incomplete records in several variables. The main missing values were found in `Delivery_person_Ratings` (1,908), `Delivery_person_Age` (1,854), `Time_Orderd` (1,731), `City` (1,200), `multiple_deliveries` (993), `Weather_conditions` (616), `Road_traffic_density` (601), and `Festival` (228).

For numeric variables, median imputation was used where appropriate. Invalid or unavailable delivery-person ages and ratings were first converted to missing values and then imputed. For categorical variables such as weather, traffic, festival, and city, missing values were replaced with the category "Unknown".

The original `Time_Orderd` field was not artificially populated when its original value was missing. Instead, the numeric feature `Order_time_minutes` was created from valid time information and used for modeling.

8.3 Duplicate and Consistency Checks

The dataset contained no exact duplicate rows. Categorical fields were standardized by trimming whitespace and mapping equivalent values to consistent forms (e.g. "fog" → "Fog", "low" → "Low").

8.4 Date and Time Processing

`Order_Date` was converted to a datetime representation. `Time_Orderd` and `Time_Order_picked` were processed to handle both HH:MM values and Excel-style time fractions. Two numeric features were created — `Order_time_minutes` and `Picked_time_minutes` — providing a consistent numerical representation of order and pickup times for machine learning.

8.5 Numeric Validity Checks

Numeric ranges were examined for delivery-person age, ratings, geographical coordinates, vehicle condition, multiple deliveries, and delivery time. Thirty-eight age records contained the invalid value 15, while 53 rating records contained the invalid value 6. These invalid values were converted to missing values before imputation. Geographical coordinates were also checked for invalid placeholder values and sign-related issues using domain-specific geographic validity rules.

8.6 Outlier Analysis

Outliers were investigated using boxplots, histograms, and the Interquartile Range (IQR) method. Predictor outliers in `Delivery_person_Ratings` and `multiple_deliveries` were capped (winsorized) to the IQR bounds so the rest of each record remained usable. `Delivery_person_Age` required no further treatment once implausible ages were already handled by the domain-validity check. Because `Time_taken` (min) is the prediction target, capping it would have distorted the ground truth, so the 270 rows with an outlier target value were removed instead of capped.

8.7 Final Validation

After preprocessing, the cleaned dataset used for EDA and modeling contains 45,314 rows and 22 columns — 45,584 original rows minus the 270 rows removed for an outlier target value, with two columns added (Order_time_minutes and Picked_time_minutes). The final validated delivery-person age range is 18–50, the rating range is 1–5, and Time_taken (min) ranges from 10 to 54 minutes, with a median of 26 minutes.

9. Exploratory Data Analysis (EDA)

Before developing the machine learning models, exploratory data analysis (EDA) was performed on the cleaned dataset (45,314 records) to understand the distribution of the target variable, examine the numeric and categorical features, and identify the factors most strongly associated with delivery time.

9.1 Distribution of the Target Variable

The target variable, Time_taken (min), was examined first since it is what the models are designed to predict.

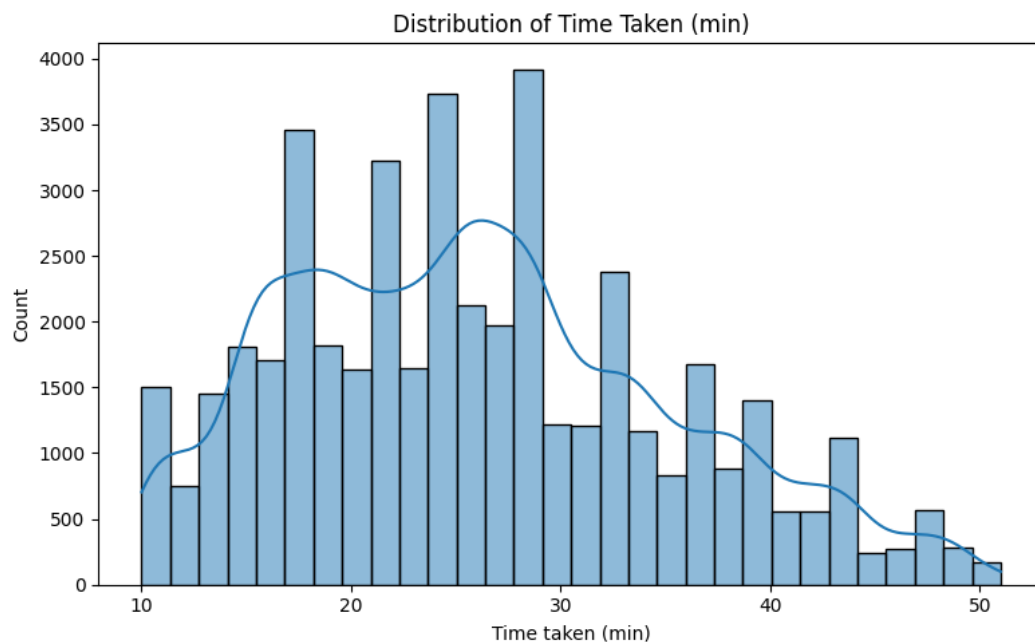


Figure 1. Distribution of Time_taken (min)

- Mean: 26.29 minutes
- Median: 26 minutes
- Standard deviation: 9.38 minutes
- Range: 10 to 54 minutes

The distribution is roughly bell-shaped with a slight right skew. No extreme outliers remain, since target outliers had already been removed during data cleaning.

9.2 Numeric Feature Distributions

The distributions of the main numeric features (Delivery_person_Age, Delivery_person_Ratings, multiple_deliveries, Order_time_minutes, and Picked_time_minutes) were also inspected. Delivery_person_Ratings is concentrated between 3.9 and 5.0 after outlier capping, Delivery_person_Age is spread fairly evenly across the working-age range, and multiple_deliveries is dominated by small values (0-2), reflecting that most drivers handle only a few parallel deliveries.

9.3 Correlation Between Numeric Features and Delivery Time

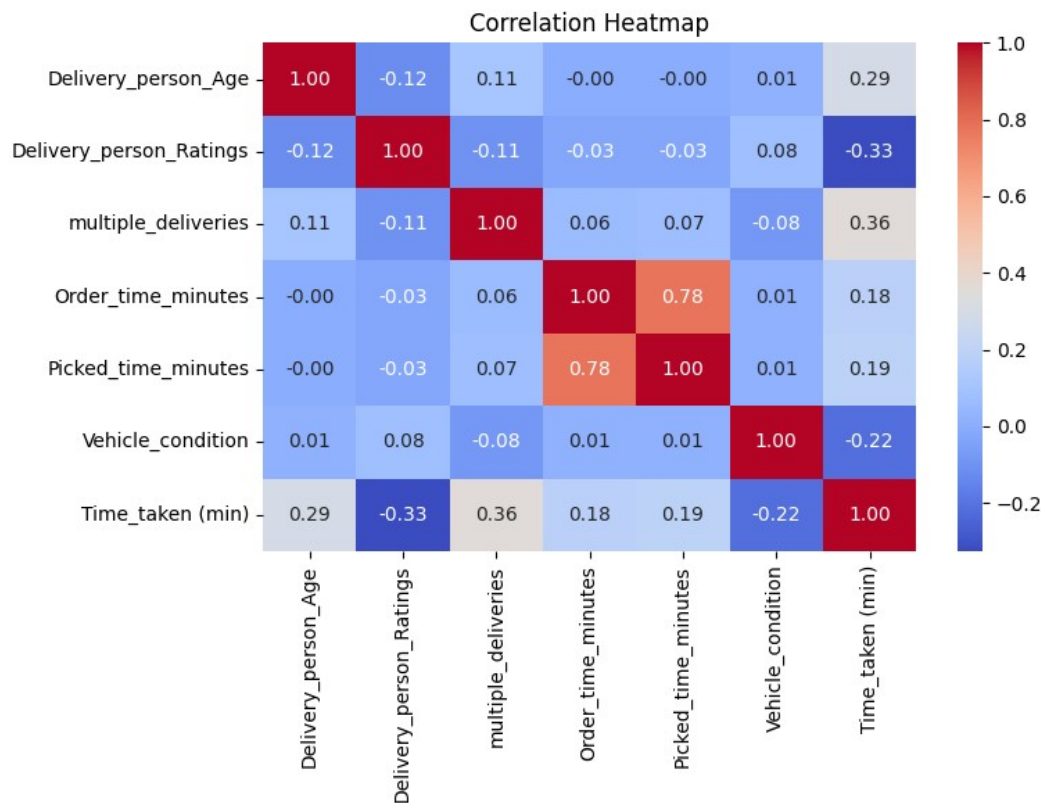


Figure 2. Correlation heatmap of numeric features with Time_taken (min)

The correlation heatmap highlights the following relationships with Time_taken (min):

- multiple_deliveries and Delivery_person_Age show the strongest positive correlations — drivers handling more parallel orders, and slightly older drivers, tend to take longer.
- Delivery_person_Ratings shows a negative correlation — higher-rated drivers tend to deliver faster.
- Order_time_minutes and Picked_time_minutes show weaker positive correlations with delivery time but are strongly correlated with each other, which is a mild multicollinearity signal worth keeping in mind for the Linear Regression model.
- Vehicle_condition is negatively correlated with delivery time.

9.4 Categorical Feature Frequencies

The frequency of each category was reviewed for Weather_conditions, Road_traffic_density, Type_of_order, Type_of_vehicle, Festival, and City. As an example, Road_traffic_density is dominated by Low and Jam traffic, followed by Medium and High, with only a small Unknown category:

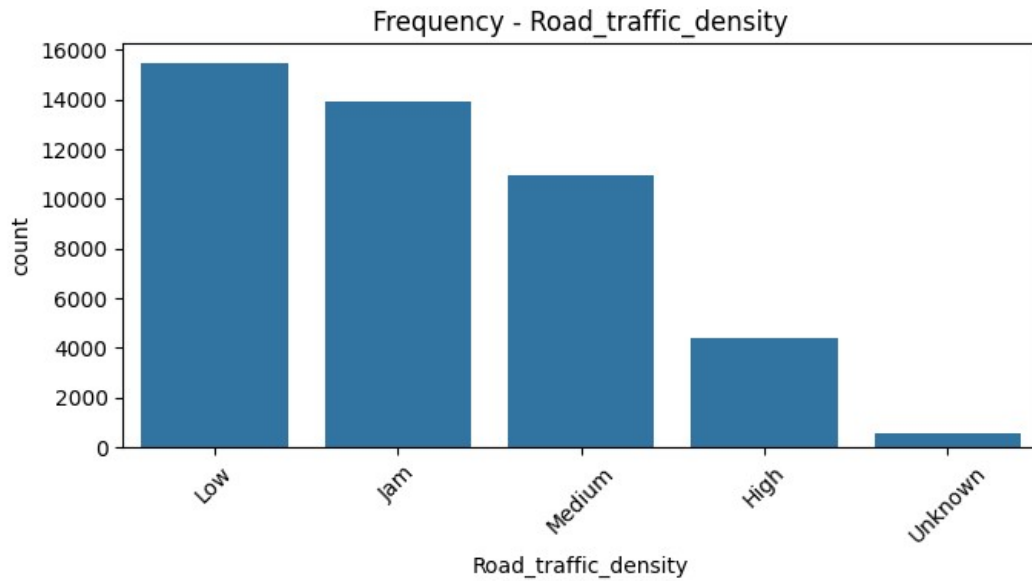


Figure 3. Frequency of Road_traffic_density categories

9.5 Delivery Time vs. Categorical Features

Box plots were used to compare Time_taken (min) across the main categorical drivers. Since a box plot's whiskers are computed per category, points that were outliers only within their own category were removed for these plots; the underlying modeling data was left untouched.

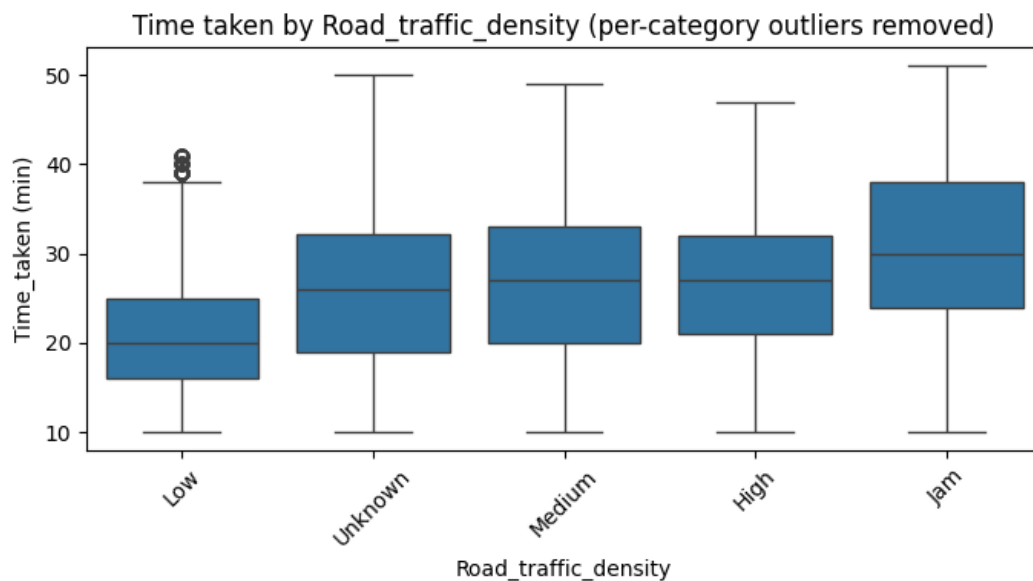


Figure 4. Time taken by Road_traffic_density

Jam traffic is associated with the highest median delivery time (around 30 minutes), compared to around 20 minutes for Low traffic — traffic density is one of the clearest drivers of delivery time.

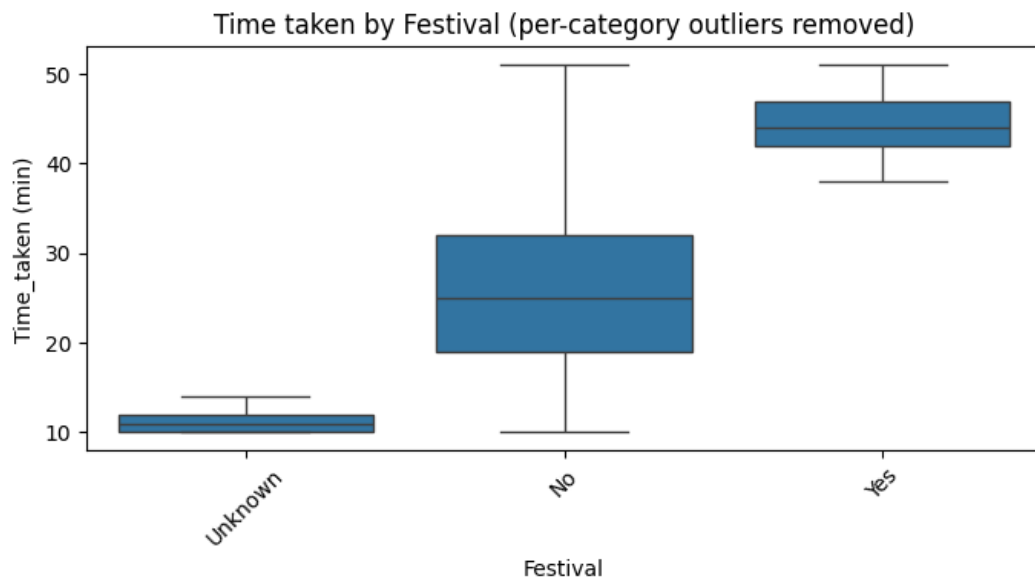


Figure 5. Time taken by Festival status

Deliveries made during a Festival show a noticeably higher median delivery time than non-festival deliveries, making Festival one of the strongest categorical predictors together with traffic density.

9.6 Delivery Time vs. Numeric Features

Scatter plots of Time_taken (min) against Delivery_person_Age, Delivery_person_Ratings, multiple_deliveries, and Order_time_minutes were also examined:

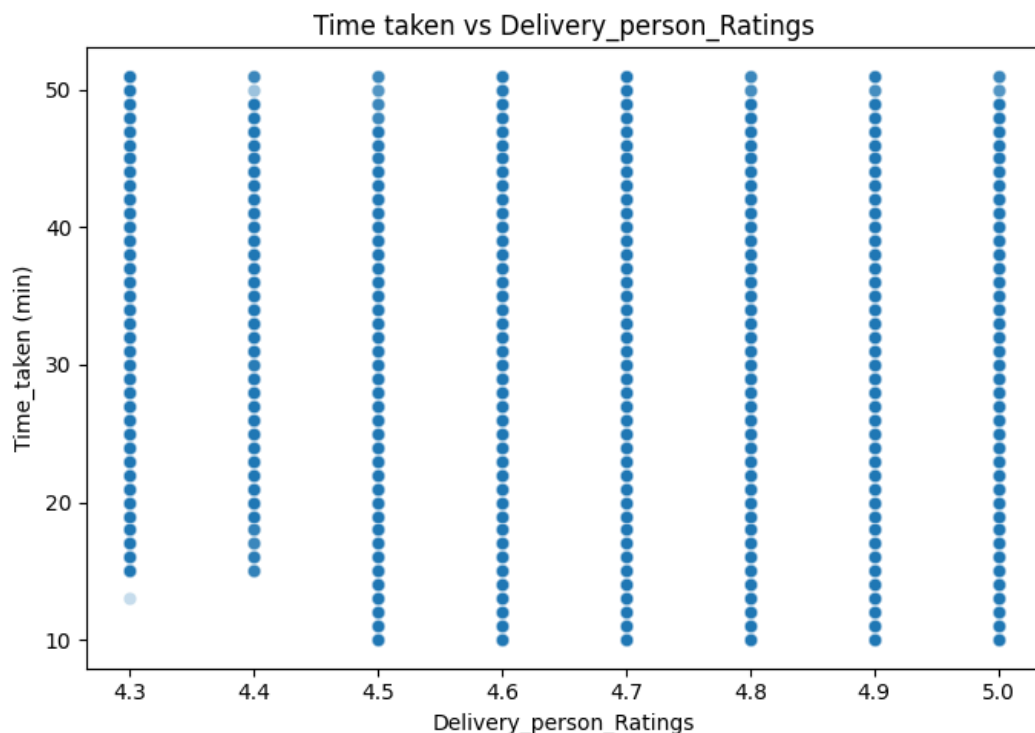


Figure 6. Time taken vs. numeric features

Higher-rated drivers show a mild tendency toward shorter delivery times, consistent with the negative correlation observed earlier. Delivery_person_Age and multiple_deliveries show weaker, but still visible, positive relationships with delivery time.

9.7 EDA Summary

- Time_taken (min) is roughly bell-shaped with a slight right skew and no extreme outliers after the earlier cleaning and outlier-treatment step.
- Road_traffic_density and Festival show the clearest separation in median delivery time — Jam traffic and Festival days are associated with longer deliveries.
- Delivery_person_Ratings shows a mild negative relationship with delivery time (higher-rated drivers tend to be a bit faster).
- Delivery_person_Age and multiple_deliveries show weaker but visible positive relationships with delivery time.
- These patterns motivate testing models that can capture non-linear effects and interactions between categorical and numeric features, in addition to a linear baseline.

10. Feature Engineering

Beyond the cleaning stage, additional features were engineered from the geographical and time-related fields to give the machine learning models more directly predictive signal than the raw coordinates and timestamps could provide on their own.

10.1 Delivery Distance (Haversine Formula)

The raw dataset provides restaurant and delivery-location coordinates (latitude/longitude) but no direct distance value. The great-circle distance between the restaurant and the delivery location was calculated using the Haversine formula, which converts the coordinate pairs into an approximate straight-line distance in kilometers:

$$distance_km = 2R \times \arcsin(\sqrt{(\sin^2(\Delta lat/2) + \cos(lat_1) \times \cos(lat_2) \times \sin^2(\Delta lon/2))})$$

where $R = 6,371$ km (Earth's radius). This single engineered feature replaces four raw coordinate columns with one directly interpretable predictor. After computing `distance_km` across the cleaned dataset, the values ranged from 1.47 km to 20.97 km, with a mean of 9.56 km and a median of 8.94 km — all physically plausible for intra-city deliveries.

10.2 Time-Based Features

Additional features were derived from the order timestamp to capture time-of-day and weekly patterns that the raw date/time fields do not expose directly to a model:

- `order_hour` — the hour of day the order was placed, derived from `Order_time_minutes`.
- `is_rush_hour` — a binary indicator set to 1 when `order_hour` falls in the lunch (12–13) or dinner (19–20) peak windows identified during EDA, and 0 otherwise.
- `order_dayofweek` — the day of the week (0 = Monday ... 6 = Sunday), derived from `Order_Date`.
- `is_weekend` — a binary indicator set to 1 when `order_dayofweek` falls on Saturday or Sunday.

10.3 Restaurant Preparation Time

A `prep_time_minutes` feature was derived as the difference between `Picked_time_minutes` and `Order_time_minutes`, representing how long the restaurant took to hand the order to the delivery person. Orders placed shortly before midnight and picked up shortly after were handled with a wraparound correction (adding 24 hours) so the resulting duration is never negative.

10.4 Final Feature Set

The final feature set used by all three regression models consists of 16 features:

Type	Features
Numeric	Delivery_person_Age, Delivery_person_Ratings, distance_km, order_hour, is_rush_hour, order_dayofweek, is_weekend, Vehicle_condition, multiple_deliveries, prep_time_minutes
Categorical	Weather_conditions, Road_traffic_density, Type_of_order, Type_of_vehicle, Festival, City

Categorical features are one-hot encoded and missing values are imputed inside a scikit-learn Pipeline (median for numeric, most-frequent for categorical), so preprocessing is reproducible and no information from the test set leaks into training.

11. Machine Learning Models

All three models were trained and evaluated on the same 80/20 train-test split (`random_state = 42`) of the 45,314-row cleaned dataset, giving 36,251 training records and 9,063 test records, and the same 16-feature set described in Section 10. This ensures the models are directly comparable on identical data.

11.1 Linear Regression (Baseline Model)

A Linear Regression model was trained as a baseline comparison model, using the same train/test split and target (`Time_taken (min)`) as the other regression models.

11.1.1 Preprocessing

- Numeric features were median-imputed and then standardized (mean 0, standard deviation 1) using `StandardScaler`, since Linear Regression — unlike a Decision Tree or Random Forest — is sensitive to feature scale.
- Categorical features were imputed with the most frequent value and one-hot encoded, using the same encoding approach as the tree-based models.
- Preprocessing and modeling were combined into a single scikit-learn Pipeline to keep the workflow reproducible and avoid data leakage.

11.1.2 Performance on the Test Set

Metric	Value
MAE	4.771 minutes
RMSE	6.001 minutes
R ²	0.583

On average, the model's delivery-time predictions are within about 4.8 minutes of the actual value, and the model explains roughly 58% of the variance in delivery time. Predictions cluster reasonably well around the perfect-prediction diagonal but the model tends to compress extreme values — it slightly overestimates very short delivery times and underestimates very long ones. This is consistent with a linear model's difficulty capturing the sharp, non-linear jumps in delivery time caused by factors such as Jam traffic or Festival days.

11.2 Decision Tree Regression

A Decision Tree Regressor was developed as a second machine learning technique for predicting `Time_taken (min)`, using feature-based splits to produce numerical delivery-time predictions.

11.2.1 Baseline and Tuning

The baseline Decision Tree (default parameters) was evaluated first, then `GridSearchCV` with 3-fold cross-validation was used to tune `max_depth`, `min_samples_split`, and `min_samples_leaf`. The best configuration found was `max_depth = 12`, `min_samples_split = 2`, `min_samples_leaf = 5`, with a best cross-validation RMSE of 4.328.

11.2.2 Performance on the Test Set

Model	MAE	RMSE	R ²
Baseline Decision Tree	4.262	5.655	0.630
Tuned Decision Tree	3.378	4.333	0.782

Tuning substantially improved the model: MAE dropped from 4.262 to 3.378 minutes and R^2 rose from 0.630 to 0.782 on the untouched test set. This shows the Decision Tree captures meaningful non-linear structure that the Linear Regression baseline could not.

11.3 Random Forest Regression (Final Model)

A Random Forest Regressor was trained as the main model, using the same features, train/test split, and target as the Decision Tree and Linear Regression models. As an ensemble of decision trees, it is expected to generalize better than a single tree while remaining scale-invariant, so no feature scaling was applied — consistent with the Decision Tree pipeline.

11.3.1 Baseline and Hyperparameter Tuning

The baseline Random Forest (default parameters, `random_state = 42`) was evaluated first, achieving MAE = 3.243, RMSE = 4.129, $R^2 = 0.802$ — already outperforming the tuned Decision Tree. GridSearchCV with 3-fold cross-validation was then used to tune `n_estimators` ({100, 200, 300}), `max_depth` ({8, 12, 16, None}), and `min_samples_leaf` ({1, 2, 5}), optimizing for RMSE. The best configuration found was `n_estimators = 200`, `max_depth = 12`, `min_samples_leaf = 2`, with a best cross-validation RMSE of 4.087.

11.3.2 Final Performance on the Test Set

Metric	Value
MAE	3.219 minutes
RMSE	4.080 minutes
R^2	0.807

The tuned Random Forest achieved the best result of all three models: predictions are within about 3.2 minutes of the actual delivery time on average, and the model explains roughly 81% of the variance in delivery time.

11.3.3 Feature Importance

Impurity-based feature importance from the tuned Random Forest identifies `Delivery_person_Ratings`, `Road_traffic_density (Low)`, `distance_km`, `multiple_deliveries`, and `Delivery_person_Age` as the strongest predictors:

Feature	Importance
<code>Delivery_person_Ratings</code>	0.231
<code>Road_traffic_density = Low</code>	0.141
<code>distance_km</code>	0.112
<code>multiple_deliveries</code>	0.111
<code>Delivery_person_Age</code>	0.092
<code>Vehicle_condition</code>	0.079
<code>Weather_conditions = Sunny</code>	0.075
<code>Weather_conditions = Fog</code>	0.044
<code>Weather_conditions = Cloudy</code>	0.043
<code>Road_traffic_density = Jam</code>	0.013

Note: Impurity-based importance can be biased toward high-cardinality one-hot columns and does not always match the direction of effect seen in the EDA (for example, `Road_traffic_density = Jam` ranks low here despite showing a clear effect on median delivery time in Section 9.5). A permutation-importance analysis on the held-out test set would give a more robust ranking and is recommended as a follow-up refinement.

11.4 Model Comparison and Selection

All three models were evaluated on the identical test set, allowing a direct, fair comparison:

Model	MAE	RMSE	R ²
Linear Regression	4.771	6.001	0.583
Decision Tree (Tuned)	3.378	4.333	0.782
Random Forest (Tuned)	3.219	4.080	0.807

The tuned Random Forest achieved the lowest MAE and RMSE and the highest R² of the three models, and was therefore selected as the final model for deployment. The ranking is consistent with the EDA finding that delivery time depends on interactions between categorical conditions (traffic, festival) and numeric features (distance, ratings) that a linear model cannot represent, but that tree-based ensembles capture well. The trained model is stored as `rf_delivery_time_model.pkl` and the complete modeling workflow is available in `notebooks/zomato_food_delivery.ipynb`.

12. Streamlit Application

The final trained Random Forest model was integrated into an interactive Streamlit web application, allowing a non-technical user to obtain a delivery-time estimate without touching the notebook or the model file directly.

12.1 Application Inputs

The application collects the same 16 features the model was trained on, grouped into three sections:

Section	Inputs
Delivery person	Age, rating, vehicle condition
Order details	Order date, order time, restaurant preparation time, number of multiple deliveries, order type, vehicle type
Conditions	Distance to delivery location, weather conditions, road traffic density, festival day, city type

Time-based features (`order_hour`, `is_rush_hour`, `order_dayofweek`, `is_weekend`) are derived automatically from the order date and time entered by the user, using the same logic applied during feature engineering (Section 10.2), so the inputs the model receives are consistent with how it was trained.

12.2 Prediction Output

Clicking “Predict delivery time” passes the assembled feature row through the saved scikit-learn Pipeline — which applies the same imputation and one-hot encoding used during training before calling the Random Forest — and displays the estimated delivery time in minutes. An expandable panel (“See the exact inputs sent to the model”) shows the exact feature values used for that prediction, which is useful both for debugging and for demonstrating the model’s reasoning during the live presentation.

12.3 Deployment

The application is deployed on Streamlit Community Cloud and is publicly accessible at:

<https://zomato-food-delivery-time-prediction-project-hu6scvwmeu88brhbc.streamlit.app/>

The application source code (`app.py`), the trained model file (`rf_delivery_time_model.pkl`), and the dependency list (`requirements.txt`) are included in the project’s GitHub repository.

13. Power BI Dashboard

The project includes an interactive four-page Power BI dashboard designed to provide business-level insights into food delivery operations, moving from high-level performance monitoring to deeper operational and root-cause analysis. The dashboard is organized into four pages: Executive Overview, Operational Performance, Root Cause Analysis, and Geographic Analysis.

13.0 Intended Audience

The dashboard is designed for operations managers and dispatch planners who need to monitor delivery performance, identify conditions associated with slow deliveries, and decide where to focus resources (staffing, vehicle allocation, or route planning) across cities and time periods.

13.1 Data Model

The underlying data model contains two tables: `Zomato_Cleaned_Dareen`, the main delivery fact table holding delivery-level operational data (city, traffic, weather, vehicle information, festival status, ratings, coordinates, and `Time_taken`), and `DateTable`, a dedicated date dimension used for time filtering and the monthly delivery-time trend.

The intended relationship is a one-to-many, date-dimension-to-fact relationship: `DateTable` (one row per calendar date) filters `Zomato_Cleaned_Dareen` (many delivery records per date). Selecting a date range on the Date slicer filters the fact table, which then recalculates every measure driving the dashboard's cards, charts, and map.

13.2 DAX Measures

Five reusable DAX measures were built on `Zomato_Cleaned_Dareen` and power every KPI card and chart in the dashboard:

Measure	Definition	Purpose
Total Orders	<code>COUNTROWS('Zomato_Cleaned_Dareen')</code>	Order volume KPI; reused across city, geographic, and performance-distribution visuals.
Avg Delivery Time	<code>AVERAGE('Zomato_Cleaned_Dareen'[Time_taken (min)])</code>	Main operational-efficiency KPI, used in nearly every visual on the dashboard.
Median Delivery Time	<code>MEDIAN('Zomato_Cleaned_Dareen'[Time_taken (min)])</code>	Less sensitive to extreme delivery times; compared against the average to detect skew.
Avg Delivery Rating	<code>AVERAGE('Zomato_Cleaned_Dareen'[Delivery_person_Ratings])</code>	Service-quality indicator, used in KPI cards and the geographic map tooltips.
Slow Delivery %	<code>DIVIDE(CALCULATE([Total Orders], [Delivery Performance]="Slow"), [Total Orders])</code>	Share of orders classified as slow, in the current filter context; <code>DIVIDE</code> avoids a divide-by-zero error.

Because each of these is a DAX measure rather than a static calculated value, every number on the dashboard recalculates automatically as the user changes the City, Traffic, Weather, Vehicle Type, or Date slicers — the same five measures serve the entire dataset, a single city, a single traffic condition, or any combination, without duplicating logic.

13.3 Page 1 — Executive Overview

Provides management with a quick snapshot of delivery performance, focusing on the most important KPIs and high-level comparisons.

KPI cards: Total Orders, Average Delivery Time, Median Delivery Time, Slow Delivery %, Average Delivery Rating — together covering volume, speed, distribution, and service quality.

Visualizations: a delivery-time trend line by month; a bar chart of total orders by city; a donut chart of the delivery-performance distribution; a column chart comparing average delivery time across traffic-density levels.

Slicers: Vehicle type, Date, Traffic, City, Weather.

13.4 Page 2 — Operational Performance

Investigates the operational conditions that may influence delivery efficiency, moving the analysis from “How are we performing?” toward “Which conditions are associated with better or worse performance?”

- Traffic Analysis — average delivery time compared across Road Traffic Density categories.
- Weather Analysis — average delivery time compared across Weather Conditions.
- Vehicle Type Analysis — average delivery time by Type of Vehicle.
- Vehicle Condition Analysis — average delivery time by Vehicle Condition.
- Multiple Deliveries Analysis — average delivery time by number of Multiple Deliveries handled per trip.
- Festival Analysis — average delivery time compared between Festival and non-Festival days.

13.5 Page 3 — Root Cause Analysis

Goes beyond category comparisons to explore relationships between delivery time and numerical variables, aiming to identify patterns and possible contributing factors without assuming causation.

- Delivery Person Age vs. Time Taken — scatter plot exploring trends, clusters, and outliers.
- Delivery Person Rating vs. Time Taken — scatter plot investigating whether rating is associated with delivery efficiency.
- Pickup Delay vs. Time Taken — scatter plot separating restaurant-side delay from the delivery journey itself.
- Traffic × Weather Matrix — investigates combinations of environmental conditions rather than viewing them independently.
- Traffic × Vehicle Type Matrix — evaluates whether vehicle performance changes under different traffic conditions.

13.6 Page 4 — Geographic Analysis

Examines how order volume and delivery performance differ across locations, which may reflect traffic patterns, order density, restaurant distribution, and vehicle mix.

- Average Delivery Time by City and Total Orders by City — bar charts distinguishing high-volume operational pressure from location-specific performance issues.
- City × Traffic Matrix and City × Vehicle Type Matrix — investigate whether city-level differences relate to traffic conditions or vehicle mix.

- Geographic Map — uses restaurant latitude/longitude together with City, Total Orders, Average Delivery Time, and Average Delivery Rating; point size represents Total Orders while tooltips surface performance detail.

13.7 Interactive Functionality and Visualization Strategy

The dashboard uses slicers for Date, City, Road Traffic Density, Weather Conditions, and Type of Vehicle, letting users dynamically explore specific cities, conditions, or vehicle categories without creating separate reports. Each visual type was selected to match its analytical question: cards for high-level KPIs, a line chart for the time trend, bar/column charts for categorical comparisons, a donut chart for performance composition, scatter plots for numeric relationships, matrices for multi-dimensional comparisons, and a map for geographic distribution.

13.8 Business Value

The analytical journey — Executive Overview → Operational Performance → Root Cause Analysis → Geographic Analysis — demonstrates how Power BI can transform operational data into a structured decision-support tool rather than simply displaying raw information. The dashboard can support operational optimization, resource allocation, vehicle management, delivery planning, customer-experience monitoring, and geographic strategy for the operations and dispatch teams it was designed for.

The dashboard file is available in the project repository at `dashboard/zomato final.pbix`.

14. Team Contributions

Team Member	Responsibilities
Menna Khaled	Feature engineering (Haversine distance, time-based features), main Random Forest model and hyperparameter tuning, model comparison and selection, Streamlit application development and deployment, project coordination and final report integration.
Abdelrahman Mahmoud	Data preprocessing support, full exploratory data analysis, baseline Linear Regression model.
Dareen Alaa	Data cleaning lead — missing values, invalid values, and outlier handling; Decision Tree Regression model.
Salma Tarek	Power BI data modeling, DAX measures, and relationship structure between the fact and date tables.
Moureen Ramy Kelada	Complete four-page Power BI dashboard development (Executive Overview, Operational Performance, Root Cause Analysis, Geographic Analysis).

15. Limitations

Machine learning predictions depend on the quality and characteristics of the available training data. Several limitations should be kept in mind when interpreting the results of this project:

- Actual delivery times may be affected by factors not represented in the dataset, such as unexpected traffic incidents, restaurant-side operational issues, sudden weather changes, or driver availability.
- The Random Forest's impurity-based feature importance does not always agree with the direction of effect seen in the EDA (Section 11.3.3) — a permutation-importance analysis would give a more reliable ranking.
- Restaurant preparation time (`prep_time_minutes`) is only known after an order has been picked up, so the deployed Streamlit application asks the user to estimate it rather than measuring it directly; predictions are only as reliable as that estimate.
- The dataset covers a fixed historical period and specific cities; performance may not generalize to other regions, seasons, or delivery-platform operating models without retraining.
- Distance is calculated as a straight-line (Haversine) distance rather than an actual road-network route distance, so it may understate real travel distance in areas with indirect road layouts.
- Median imputation for missing numeric values (`Delivery_person_Age`, `Delivery_person_Ratings`, `multiple_deliveries`, and geographic coordinates) was computed once on the full cleaned dataset during preprocessing, before the train/test split was created. This means the imputed medians were influenced by values in what later became the test set — a mild form of data leakage. Its practical impact is likely small, since the affected columns hold relatively few missing values and imputing with the training-set median alone would probably produce a very similar value, but a stricter pipeline would compute imputation statistics after splitting, fitting the imputer on the training set only.

Therefore, model predictions should be considered estimates rather than guaranteed delivery times.

16. Conclusion

This project demonstrates a complete, end-to-end data science workflow applied to a real-world operational problem: predicting food delivery time from the Zomato Delivery Operations Analytics Dataset. Starting from a raw dataset of 45,584 orders, the team cleaned and validated the data, explored the relationships between delivery time and operational, environmental, and time-related factors, and engineered features — most notably geographic distance via the Haversine formula and several time-based indicators — that captured meaningful predictive signal beyond the raw fields.

Three regression models were trained and fairly compared on an identical test set. The tuned Random Forest Regressor outperformed both the Linear Regression baseline and the Decision Tree, achieving an MAE of 3.22 minutes and an R^2 of 0.807, and was selected as the model behind the deployed Streamlit application. The accompanying four-page Power BI dashboard translates the same dataset into an interactive decision-support tool for operations teams, moving from an executive summary through operational and root-cause analysis to a geographic view.

Together, the trained model, the live Streamlit application, and the Power BI dashboard form a complete prediction-and-analysis system that satisfies the project's original business question: operational, geographical, environmental, and time-related factors can be used to predict food delivery time with a useful degree of accuracy, and the resulting insights can support real operational decisions.

17. References

Badole, S. Zomato Delivery Operations Analytics Dataset [Data set]. Kaggle. Retrieved September 18, 2026, from <https://www.kaggle.com/datasets/saurabhbadole/zomato-delivery-operations-analytics-dataset>

18. Appendices

Appendix A — Project Links

Resource	Link
GitHub Repository	https://github.com/mennakhaled456/Zomato-Food-Delivery-Time-Prediction-project
Live Streamlit Application	https://zomato-food-delivery-time-prediction-project-hu6scvwmneu88brhbc.streamlit.app/
Power BI Dashboard	dashboard/zomato final.pbix (in the GitHub repository)
Project Notebook	notebooks/zomato_food_delivery.ipynb (in the GitHub repository)